

Semantic Search Engine

Amazon SDE Interview Prep - Code-Verified Project Notes

Important scope: this repository implements a Python/FastAPI backend, Qdrant vector search, clustering utilities, an in-memory semantic cache, and smoke scripts. It does not implement a frontend, authentication, persistent distributed caching, formal unit tests, or production benchmarks.

1. Project Summary

Problem: keyword search misses intent when users phrase the same idea differently. This project retrieves documents by semantic meaning.

What it does: preprocesses 20 Newsgroups documents, embeds them with Sentence Transformers, stores vectors in Qdrant, exposes a FastAPI search API, and reuses semantically similar previous query results through a cache.

What I built: document preprocessing, embedding, vector storage, semantic retrieval, PCA/GMM clustering, cluster-aware semantic cache, cache metrics, and smoke scripts.

45-second answer

I built a semantic search engine for context-aware document retrieval. The system preprocesses 20 Newsgroups documents, converts them into 384-dimensional Sentence Transformer embeddings, and stores those vectors in Qdrant for similarity search. I exposed retrieval through a FastAPI backend where a user submits a natural-language query and receives the most semantically relevant documents. To reduce redundant vector searches, I added an in-memory semantic cache: incoming queries are embedded, assigned to a dominant GMM cluster after PCA reduction, and compared only against cached queries in the same semantic region. If similarity is above a threshold, the API returns cached results; otherwise it queries Qdrant and stores the new result.

2. Architecture

```
Indexing/setup path:
20 Newsgroups docs -> preprocess/normalize -> SentenceTransformer embeddings -> Qdrant
semantic_documents collection

API query path:
User/client -> FastAPI POST /query -> SemanticCache -> embed query -> PCA + GMM cluster -> cache
lookup -> hit returns cached result; miss calls SemanticSearcher -> Qdrant vector search -> JSON
response
```

One complete request

- User sends POST /query with a natural-language query.
- FastAPI validates the body with QueryRequest.
- The route calls SemanticCache.search with top_k=5.
- The cache embeds the query and predicts a dominant cluster using PCA + GMM.
- It compares against cached query embeddings in the same cluster using cosine similarity.
- On a cache hit, it returns the cached retrieval result.
- On a miss, SemanticSearcher embeds/searches Qdrant, caches the result, and the API returns structured JSON.

3. Tech Stack

Tech	Used in this project	Why chosen	Alternative
Python	All backend, ML, scripts	Strong ML/API ecosystem	Java, Node.js

Tech	Used in this project	Why chosen	Alternative
FastAPI	Routes for /query, /cache/stats, /cache	Typed, lightweight, Pydantic-friendly	Flask, Django REST
Pydantic	Request/response schemas	Validation and clear contracts	Marshmallow, dataclasses
Sentence Transformers	all-MiniLM-L6-v2 embeddings	Lightweight semantic vectors	OpenAI embeddings, TF-IDF, BERT
Qdrant	Stores/searches vectors + payloads	Native vector similarity search	Pinecone, Weaviate, FAISS, pgvector
PCA	Reduces vectors before GMM	Stabilizes clustering	UMAP, no reduction
GMM	Soft semantic clusters	Probabilistic assignments	KMeans, HDBSCAN
NumPy	Cosine similarity and matrices	Fast vector math	PyTorch tensors
Docker Compose	Runs local Qdrant	Reproducible local DB	Managed Qdrant, native install

4. Important Code

File/function	What it does	Why to know it
app/routes.py:get_semantic_cache, query_documents	Builds store, embedder, searcher, GMM clusterer, semantic cache; exposes API routes.	Main request entry point.
cache/semantic_cache.py:SemanticCache.search	Embeds query, predicts cluster, checks semantic cache, falls back to searcher.	Core optimization logic.
core/qdrant_store.py:QdrantDocumentStore	Creates collection, upserts vectors, searches, scrolls points, updates payload.	Database boundary.
core/embedder.py:TextEmbedder	Loads all-MiniLM-L6-v2 and returns normalized embeddings.	Embedding layer.
clustering/gmm_cluster.py:cluster_documents/build_pieces	Loads Qdrant vectors, applies PCA, trains GMM, stores cluster payloads.	Clustering/cache support.

5. Technical Decisions

- **Embeddings over keywords:** I chose embeddings because users may phrase the same intent differently; semantic search matches meaning, not just exact terms.
- **Qdrant over relational DB:** the core operation is nearest-neighbor vector search, so a vector database is a better fit than normal SQL indexes.
- **In-memory semantic cache:** simple and fast for a prototype, but not persistent or shared across processes.
- **PCA before GMM:** reducing 384-dimensional vectors makes GMM more stable and less overconfident.
- **GMM over KMeans:** GMM gives probabilities, which better represents ambiguous documents spanning multiple topics.

6. Interview Questions

1. How does your project work end-to-end?

Answer: Documents are cleaned, embedded, stored in Qdrant, then queries are embedded and searched by vector similarity through FastAPI.

Follow-up: Where does caching happen?

Follow-up answer: In SemanticCache.search before Qdrant is called.

2. Why semantic search instead of keyword search?

Answer: It handles different wording with similar meaning.

Follow-up: What is the downside?

Follow-up answer: It can return related but irrelevant results if embeddings are too broad.

3. Why Qdrant?

Answer: It is designed for vector similarity search and stores metadata payloads with vectors.

Follow-up: Could you use Postgres?

Follow-up answer: Yes, with pgvector, especially if relational joins were important.

4. What embedding model did you use?

Answer: all-MiniLM-L6-v2, producing 384-dimensional normalized vectors.

Follow-up: Why normalize embeddings?

Follow-up answer: It makes cosine similarity consistent and easier to compare.

5. Explain the cache.

Answer: It stores previous query embeddings and results; new queries reuse results if similarity exceeds the threshold.

Follow-up: What is the threshold?

Follow-up answer: The API default is 0.70.

6. Why cluster-aware cache?

Answer: It compares only within the same semantic region, reducing comparisons and cross-topic false hits.

Follow-up: Is it guaranteed correct?

Follow-up answer: No. Cluster assignment can be wrong; it is a tradeoff.

7. Why PCA before GMM?

Answer: GMM in 384 dimensions can be unstable; PCA keeps major signal while reducing noise.

Follow-up: Why not t-SNE?

Follow-up answer: t-SNE is mainly for visualization, not stable production transformation.

8. Why GMM instead of KMeans?

Answer: GMM gives soft probabilities, better for documents spanning multiple topics.

Follow-up: What is the cost?

Follow-up answer: It is more complex and sensitive to distribution assumptions.

9. What happens on Qdrant failure?

Answer: The /query route catches exceptions and returns HTTP 503.

Follow-up: Is error handling complete?

Follow-up answer: No. Production should add logs, health checks, retries, and startup validation.

10. Is the cache scalable?

Answer: Not as written; it is in-memory per process.

Follow-up: How would you scale it?

Follow-up answer: Move cache entries to Redis or Qdrant, shard by cluster, and add TTL/eviction.

11. How are documents indexed?

Answer: A smoke script loads 500 local documents, embeds them, recreates the Qdrant collection, and upserts vectors.

Follow-up: Does the API index documents?

Follow-up answer: No. Indexing is a separate setup script.

12. What database schema do you have?

Answer: Qdrant collection semantic_documents stores vectors plus payload: document id, text, category, and optional cluster metadata.

Follow-up: Where is it defined?

Follow-up answer: In QdrantDocumentStore.upsert_documents and the clustering payload updates.

13. What security exists?

Answer: Only Pydantic min-length validation for the query input.

Follow-up: What is missing?

Follow-up answer: Auth, rate limits, input length limits, CORS policy, and config/secrets management.

14. How did you evaluate the cache threshold?

Answer: A helper computes TP/FP/TN/FN, precision, recall, and effectiveness over labeled query pairs.

Follow-up: Is there a real benchmark?

Follow-up answer: No, only a small scripted experiment.

15. What would you improve first?

Answer: Persistent/distributed cache, real tests, configuration, health checks, and benchmark metrics.

Follow-up: Why tests first?

Follow-up answer: Search quality and cache behavior can regress silently.

7. Resume Verification

Resume claim	Code support	How to defend	Flag
Built semantic search using Sentence Transformer embeddings and Qdrant.	core/embedder.py, core/qdrant_store.py, core/search.py	I convert text into normalized embeddings, store them in Qdrant, and retrieve nearest documents for natural-language queries.	Accurate
Developed a FastAPI backend.	app/main.py, app/routes.py, app/schemas.py	The API validates requests, invokes cache/search, and returns structured results.	Accurate
PCA and Gaussian Mixture Model-based semantic cache.	cache/semantic_cache.py, clustering/gmm_cluster.py	I train PCA + GMM from stored vectors and use the predicted cluster to narrow semantic cache lookup.	Accurate, but mention in-memory and startup-trained
Optimize similarity search and reduce redundant queries.	cache/semantic_cache.py, cache/cache_metrics.py	A cache hit returns stored results instead of calling Qdrant.	Mechanism exists; no production benchmark proving latency reduction

Incomplete or not implemented

- No frontend is implemented.
- No authentication, authorization, rate limiting, CORS configuration, or input max-length enforcement beyond min_length=1.
- Cache is in memory only: not persistent, not shared across workers, no TTL, no eviction policy.
- Indexing is not an API endpoint; it is done through scripts.
- Tests are smoke/demo scripts, not a formal unit/integration test suite.
- There is no measured production benchmark showing actual latency or cost reduction.

8. Final Cheat Sheet

Problem: Keyword search misses intent; semantic search retrieves documents by meaning.

Architecture: Clean docs -> embed -> store in Qdrant -> FastAPI /query -> semantic cache -> Qdrant on miss -> JSON response.

Tech stack: Python, FastAPI, Pydantic, Sentence Transformers, Qdrant, scikit-learn PCA/GMM, NumPy, Docker Compose.

My contribution: Built backend pipeline, vector store wrapper, retrieval service, cluster-aware cache, metrics, and smoke scripts.

Hardest problem: Semantic caching safely: lower thresholds increase hits but risk false positives.

3 decisions: embeddings over keywords; Qdrant over SQL; GMM over KMeans for soft clusters.

Biggest tradeoff: In-memory cache is simple and fast, but not persistent or shared across processes.

How I would scale: Redis/Qdrant-backed cache, TTL/eviction, async API, batch embeddings, managed Qdrant, monitoring, and load tests.

What I would improve: formal tests, auth/rate limiting, env config, indexing job/API, observability, and benchmarks.

Top 10 likely questions

Question	Short answer
Why embeddings?	Meaning-based retrieval.
Why Qdrant?	Native vector search.
Why FastAPI?	Typed, lightweight API.
Why PCA?	Stabilizes GMM on high-dimensional vectors.
Why GMM?	Soft cluster probabilities.
What is cached?	Query embedding, result list, cluster, timestamp.
Cache hit condition?	Same-cluster candidates plus cosine similarity above threshold.
What if cache misses?	Search Qdrant and store result.
Main limitation?	In-memory single-process cache.
Not implemented?	Frontend, auth, persistent cache, real benchmark, formal unit tests.

Best final framing: be confident about what is implemented, and be equally crisp about limitations. Amazon interviewers usually respect clear tradeoff thinking more than inflated claims.